

Arduino® UNO Q

Arduino UNO Q: Technical Learning Document ภาษาไทย

จากบอร์ดไฟกระพริบ สู่บอร์ดประมวลผลปัญญาประดิษฐ์แบบ Edge AI



ภาพปกจากสไลด์หน้า 1: Arduino UNO Q และแนวคิด “ก้าวสู่โลกที่ไร้ขอบเขต”

วันที่สร้าง: 23/05/2026

Version 1.0 - Premium Learning Edition

จัดทำในรูปแบบหนังสือเรียนภาษาไทยเพื่อการเรียนรู้และเตรียมสอบ

สารบัญ (Table of Contents)

ส่วนเอกสาร	เนื้อหาหลัก
บทนำ	ภาพรวมเอกสาร จุดประสงค์ และพื้นฐานที่ควรรู้
Chapter 1	From Blink to Think: วิวัฒนาการจากบอร์ดควบคุมสู่บอร์ด Edge AI
Chapter 2	Dual-Brain Architecture: MPU + MCU ทำงานร่วมกันอย่างไร
Chapter 3	Hardware Technical Breakdown: สเปก พอร์ต หน่วยความจำ และอินเทอร์เฟซ
Chapter 4	Expansion System: Qwiic, Modulino, Daisy-Chaining และ Carrier Board
Chapter 5	Arduino App Lab, Bricks, Python, Sketch และ AI Models
Chapter 6	AI / Embedded Workflow: Sensor Flow, Vision Flow และ Real-time Processing
Chapter 7	Applications & Engineering View: Robotics, Smart Home, Prototyping
Chapter 8	Developer Notes, Debugging, Best Practices และ Exam Prep
Final Summary	Key Takeaways, Skills Learned และหัวข้อที่ควรเรียนต่อ

วิธีอ่านเอกสารนี้

อ่าน Chapter 1-2 เพื่อเข้าใจภาพรวมและสถาปัตยกรรมก่อน จากนั้นอ่าน Chapter 3-5 เพื่อเข้าใจฮาร์ดแวร์และเครื่องมือพัฒนา แล้วใช้ Chapter 6-8 เป็นคู่มือวิเคราะห์ระบบ ออกแบบโปรเจกต์ และทบทวนก่อนสอบ

บทนำ

เอกสารนี้เกี่ยวกับอะไร

เอกสารนี้แปลงเนื้อหาจากสไลด์ Arduino UNO Q ให้กลายเป็นหนังสือเรียนภาษาไทยที่อ่านต่อเนื่องได้
เหมาะสำหรับคนที่ไม่อยากอ่านสไลด์แบบกระจัดกระจาย แต่ต้องการเข้าใจว่าบอร์ดนี้คืออะไร ทำไมถึงสำคัญ
และควรคิดเชิงวิศวกรรมอย่างไรเมื่อนำไปใช้จริง

แกนหลักของ Arduino UNO Q คือการรวม “โลกของ Linux” ที่เหมาะกับงานประมวลผลหนัก เช่น Python, Web UI, AI inference,
Docker และการเชื่อมต่อเครือข่าย เข้ากับ “โลกของ Real-time” ที่เหมาะกับการควบคุมฮาร์ดแวร์แบบเวลาต้องแม่นยำ เช่น Motor, Sensor,
GPIO, PWM และงานที่ต้องตอบสนองทันที

เหมาะกับใคร

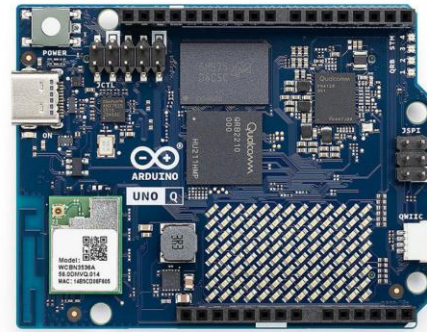
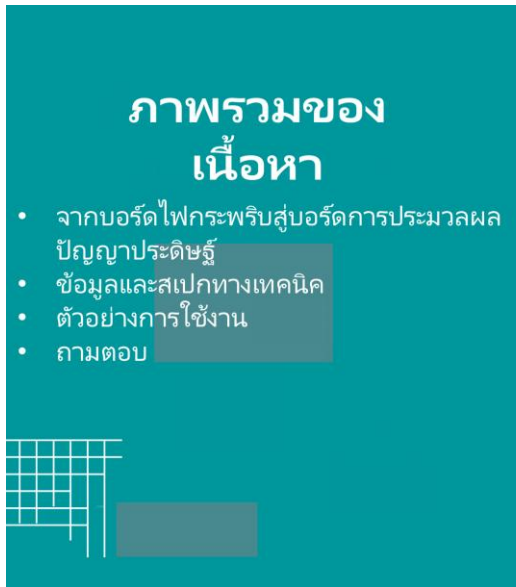
- นักศึกษาที่เรียน Embedded Systems, IoT, Robotics หรือ Computer Vision
- Developer ที่อยากเข้าใจการพัฒนา Edge AI บนฮาร์ดแวร์จริง
- วิศวกรที่ต้องออกแบบระบบที่มีทั้ง AI, Sensor, UI และ Control
- ผู้เริ่มต้นที่เคยใช้ Arduino แบบดั้งเดิม และอยากขยับไปสู่ระบบที่มี Linux และ AI

พื้นฐานที่ควรมีก่อนอ่าน

- เข้าใจพื้นฐาน Arduino เช่น Sketch, pin, sensor และ actuator
- เข้าใจภาษา Python หรือ C/C++ ระดับเบื้องต้น
- รู้จัก Linux command line บ้าง เช่น file, process, network, package
- เข้าใจแนวคิด AI inference แบบคร่าว ๆ เช่น image classification, face detection, sound recognition

แนวคิดสำคัญตั้งแต่ต้น

Arduino UNO Q ไม่ใช่แค่ Arduino UNO ที่เร็วขึ้น แต่เป็นบอร์ดแบบ Hybrid Computing ที่รวมคอมพิวเตอร์ Linux
ขนาดเล็กกับไมโครคอนโทรลเลอร์ real-time ไว้ในบอร์ดเดียว เป้าหมายคือทำให้ AI + UI + Hardware Control อยู่ใกล้กันมากขึ้น
ลดสาย ลดกล่อง ลดความซับซ้อน และพัฒนาโปรเจกต์ได้เร็วขึ้น



สไลด์หน้า 2: ภาพรวมเนื้อหาต้นฉบับที่กล่าวถึงการเปลี่ยนจากบอร์ดไฟกระพริบไปสู่บอร์ดประมวลผล AI

Chapter 1: From Blink to Think: จากไฟกระพริบสู่การคิดด้วย AI

1. Overview

คำว่า “From Blink to Think” เป็นการเปรียบเทียบเส้นทางของ Arduino จากอดีตที่เริ่มจากโปรเจกต์ง่าย ๆ เช่นทำ LED กระพริบ ไปสู่บอร์ดที่สามารถประมวลผลข้อมูล ซ้อนตรรกะระดับระบบ และรันโมเดล AI ได้บนอุปกรณ์จริง

ในอดีต Arduino ถูกมองว่าเป็นบอร์ดสำหรับเรียนรู้ input/output แต่ Arduino UNO Q พยายามยกระดับให้เป็นแพลตฟอร์มสำหรับงาน Edge Computing ที่สามารถรับข้อมูลจากเซนเซอร์ ประมวลผลด้วย AI แสดงผลผ่าน UI และควบคุมฮาร์ดแวร์แบบ real-time ในเครื่องเดียว

2. Core Concepts

- Blink คือสัญลักษณ์ของการควบคุมพื้นฐาน: เขียนโค้ด สั่ง pin เปิด/ปิด
- Think คือการเพิ่มความสามารถด้านการรับรู้ เช่น จำแนกภาพ ตรวจจับเสียง วิเคราะห์ข้อมูล และตัดสินใจ
- Edge AI คือการรัน AI ใกล้แหล่งข้อมูล ไม่ต้องส่งทุกอย่างขึ้น Cloud
- Hybrid Board คือบอร์ดที่รวม processor คนละประเภทเพื่อใช้จุดแข็งของแต่ละแบบ

3. Technical Breakdown

ถ้ามองแบบระบบ Arduino UNO Q มีบทบาทเป็นศูนย์กลางที่รับข้อมูลจาก sensor หรือ camera แล้วส่งให้ฝั่ง Linux ประมวลผลด้วย Python/AI จากนั้นส่งคำสั่งกลับไปฝั่ง MCU เพื่อควบคุมอุปกรณ์จริง เช่น motor, servo, relay หรือ LED matrix

สิ่งที่เปลี่ยนไปจาก Arduino แบบเดิมคือความสามารถด้านซอฟต์แวร์ Arduino แบบดั้งเดิมเน้น firmware และ loop() เป็นหลัก แต่ UNO Q เพิ่ม Linux Debian OS, Docker, Python, Web UI และ AI Model เข้ามา ทำให้บอร์ดใกล้เคียงคอมพิวเตอร์ Edge ขนาดเล็กมากขึ้น

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
Blink	โปรแกรมเริ่มต้นที่ทำให้ LED กระพริบ	ใช้สอนพื้นฐาน pin output และ loop
Think	การประมวลผลข้อมูลเพื่อให้ระบบตัดสินใจได้	ใช้แทนแนวคิด AI / intelligence บนบอร์ด
Edge AI	AI ที่รันบนอุปกรณ์ปลายทาง	ลด latency ลดการพึ่งพา cloud และเพิ่ม privacy
Hybrid Board	บอร์ดที่รวม MPU และ MCU	ใช้ทั้งงานประมวลผลหนักและงาน real-time

5. Architecture / Workflow

1. รับข้อมูลจาก sensor, camera หรือ input อื่น ๆ
2. ประมวลผลบน Linux ด้วย Python, AI model หรือบริการใน Docker
3. แปลงผลลัพธ์เป็นคำสั่งควบคุม เช่น เปิด relay, หมุน servo, ส่ง PWM
4. ให้ MCU ทำงานที่ต้องแม่นยำเวลา เช่นอ่านค่า sensor ต่อเนื่องหรือควบคุม motor
5. แสดงผลผ่าน Web UI, LED Matrix หรือจอภายนอก

6. Real-world Example

ตัวอย่างเช่นระบบคัดแยกวัตถุด้วยกล้อง: กล้องส่งภาพเข้าบอร์ด ฝั่ง Linux รัน image classification เพื่อดูว่าวัตถุคืออะไร จากนั้นส่งผลลัพธ์ให้ MCU ควบคุม servo เพื่อผลักวัตถุไปยังช่องที่ถูกต้อง ระบบแบบนี้ต้องการทั้ง AI และ real-time control จึงเหมาะกับบอร์ด dual-brain

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
รวมหลายงานไว้ในบอร์ดเดียว ลดจำนวนอุปกรณ์	ซับซ้อนกว่า Arduino แบบ MCU อย่างเดียว
รัน AI ได้ใกล้เคียงแหล่งข้อมูล ทำให้ตอบสนองเร็วขึ้น	ต้องเข้าใจทั้ง Linux และ embedded real-time
เหมาะกับการ prototyping เพราะมี ecosystem Arduino	งาน AI หนักมากอาจยังต้องพึ่งฮาร์ดแวร์เฉพาะทางหรือ optimization

8. Common Mistakes

- คิดว่า UNO Q คือ Arduino UNO แบบเร็วขึ้นเฉย ๆ
- เอางาน real-time ไปทำบน Linux ทั้งหมดจน timing ไม่นิ่ง
- ส่งข้อมูลภาพหรือ sensor ขึ้น cloud ทั้งหมดโดยไม่จำเป็น
- ไม่แยกหน้าที่ระหว่าง AI inference, UI และ control

9. Best Practices

- แยกงานประมวลผลหนักให้ MPU และงานควบคุมเวลาสำคัญให้ MCU
- เริ่มจาก prototype ยากก่อน เช่น read sensor -> show UI -> control output
- วัด latency จริงเสมอ โดยเฉพาะงาน robotics และ vision
- ออกแบบให้ระบบยังทำงานพื้นฐานได้แม้ network ล่ม

10. Summary

สรุปท้ายบท

หัวใจของบทนี้คือ Arduino UNO Q เปลี่ยนภาพจำของ Arduino จากบอร์ดควบคุม I/O ธรรมดาไปสู่แพลตฟอร์ม Edge AI ที่สามารถ “รับรู้ วิเคราะห์ และตอบสนอง” ได้ในอุปกรณ์เดียว



สไลด์หน้า 3: From Blink to Think แสดงแนวคิดการยกระดับ Arduino ไปสู่การประมวลผล AI

Chapter 2: Dual-Brain Architecture: MPU + MCU

1. Overview

สถาปัตยกรรมสองสมองเป็นจุดที่สำคัญที่สุดของ Arduino UNO Q เพราะบอร์ดไม่ได้ใช้หน่วยประมวลผลเพียงชนิดเดียว แต่แบ่งงานให้เหมาะกับธรรมชาติของ processor แต่ละฝั่ง

MPU หรือ Microprocessor Unit เหมาะกับการรันระบบปฏิบัติการเต็มรูปแบบ เช่น Linux, Python, AI libraries, Docker และ Web UI ส่วน MCU หรือ Microcontroller Unit เหมาะกับงานควบคุมอุปกรณ์แบบ deterministic หรือคาดเดาเวลาได้ดี

2. Core Concepts

- MPU คือสมองนักคิด ทำงานกับ Linux, AI, UI, network และ storage
- MCU คือร่างกายนักปฏิบัติ ทำงานกับ pin, sensor, PWM, interrupt และ real-time control
- Linux ให้ความยืดหยุ่น แต่ไม่รับประกัน timing แบบ hard real-time
- RTOS ช่วยให้ MCU จัดการ task ที่ต้องตอบสนองตามเวลาได้ดีกว่า loop แบบเดิม
- RPC เป็นสะพานให้สองโลกคุยกันโดยไม่ต้องรวมโค้ดทั้งหมดเข้าก้อนเดียว

3. Technical Breakdown

ในสไลด์มีการอธิบายว่า Arduino UNO Q ใช้ Qualcomm Dragonwing QRB2210 เป็น MPU และ STM32U585 เป็น MCU การแยกเช่นนี้ช่วยให้ฝั่ง Linux ทำงานที่ซับซ้อน เช่น camera capture, image classification, local dashboard และ data processing ขณะที่ฝั่ง MCU คุม servo motor หรือ sensor timing

ปัญหาที่ระบบเดิมมักเจอคือมีหลายกล่องแยกกัน เช่น กล่อง AI, กล่อง HMI, กล่อง real-time control แล้วต้องเดินสายหรือทำ network ระหว่างกัน UNO Q ลดปัญหานี้ด้วยการเอาฟังก์ชันหลักมาอยู่ในบอร์ดเดียว

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
MPU	Microprocessor Unit ที่รัน OS ได้	ใช้รัน Linux, Python, AI, Docker, Web UI
MCU	Microcontroller Unit สำหรับควบคุมฮาร์ดแวร์	ใช้ควบคุม sensor, actuator, GPIO, PWM
Linux Debian OS	ระบบปฏิบัติการ Linux เต็มรูปแบบ	ใช้จัดการไฟล์ process network package และ app
Zephyr OS	RTOS ที่ใช้กับ Arduino Core บน MCU	ใช้ควบคุมงาน real-time และ

Term	ความหมาย	ใช้ทำอะไร
		firmware
RPC	Remote Procedure Call	ใช้ให้ process หรือ processor คนละฝั่งเรียกงานกันได้

5. Architecture / Workflow

- กล้องหรือ sensor ส่งข้อมูลเข้าสู่ระบบ
- ฝั่ง MPU รับข้อมูลและเตรียมข้อมูล เช่น resize image, normalize, encode
- AI model หรือ Python script วิเคราะห์ข้อมูล
- ผลลัพธ์ถูกส่งผ่าน layer สื่อสาร เช่น RPC ไปยัง MCU
- MCU สั่ง hardware ตามผลลัพธ์ เช่น servo, motor, LED หรือ relay
- สถานะระบบถูกแสดงผ่าน Web UI / HMI / LED Matrix

6. Real-world Example

ระบบประตูอัจฉริยะ: กล้องจับภาพใบหน้า ฝั่ง MPU ทำ face detection และตรวจสอบตัวตน เมื่อผ่านเงื่อนไขจะส่งคำสั่งให้ MCU เปิด relay หรือ servo lock โดย MCU รับผิดชอบ timing และ safety ของกลไกเปิดประตู

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
แยกงานถูกประเภท ทำให้ระบบเสถียรกว่า	debug ยากขึ้นเพราะมีสอง runtime
ลดความซับซ้อนของการต่อหลายบอร์ด	ต้องจัดการ communication ระหว่าง MPU และ MCU
รองรับทั้ง AI และ real-time control ในตัว	ถ้าออกแบบ protocol ไม่ดี อาจเกิดคอขวดหรือ race condition

8. Common Mistakes

- ให้ Linux คุม motor timing โดยตรงแทน MCU
- ส่งข้อมูลภาพขนาดใหญ่ระหว่าง processor บ่อยเกินไป
- ไม่กำหนด state machine ที่ชัดเจนระหว่าง AI result และ actuator action
- ไม่ใส่ timeout หรือ fallback เมื่อฝั่งใดฝั่งหนึ่งค้าง

9. Best Practices

- ออกแบบ message ให้เล็กและชัด เช่น command, status, timestamp
- แยก safety-critical control ไว้ที่ MCU
- ใช้ logging ทั้งสองฝั่งเพื่อ debug
- ทำ watchdog หรือ heartbeat ระหว่าง MPU/MCU
- กำหนด latency budget ตั้งแต่ต้น

10. Summary

สรุปท้ายบท

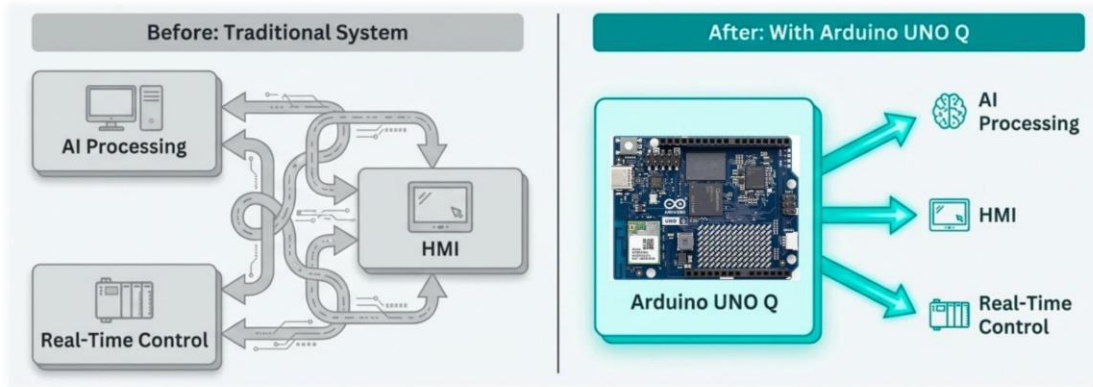
Dual-Brain Architecture คือแนวคิดที่ไม่มี processor ตัวเดียวที่เก่งทุกอย่าง จึงต้องจับคู่ “สมองประมวลผลหนัก” กับ “สมองควบคุมเวลาจริง” เพื่อสร้างระบบ embedded AI ที่ทั้งฉลาดและตอบสนองได้แม่นยำ



การผสาน “สมองนักคิด” (MPU) และ “ร่างกายนักปฏิบัติ” (MCU) จากอดีตที่เคยเป็นเรื่องซับซ้อน ต้องใช้เครื่องมือและโลกในการพัฒนาที่แตกต่างกันสองชุด และต้องเสียเวลาไปกับการเชื่อมต่อ แทนที่จะได้สร้างสรรค์สิ่งใหม่

สไลด์หน้า 5: ภาพเปรียบเทียบ Linux World กับ Real-Time World และความท้าทายในการเชื่อมสองโลกเข้าด้วยกัน

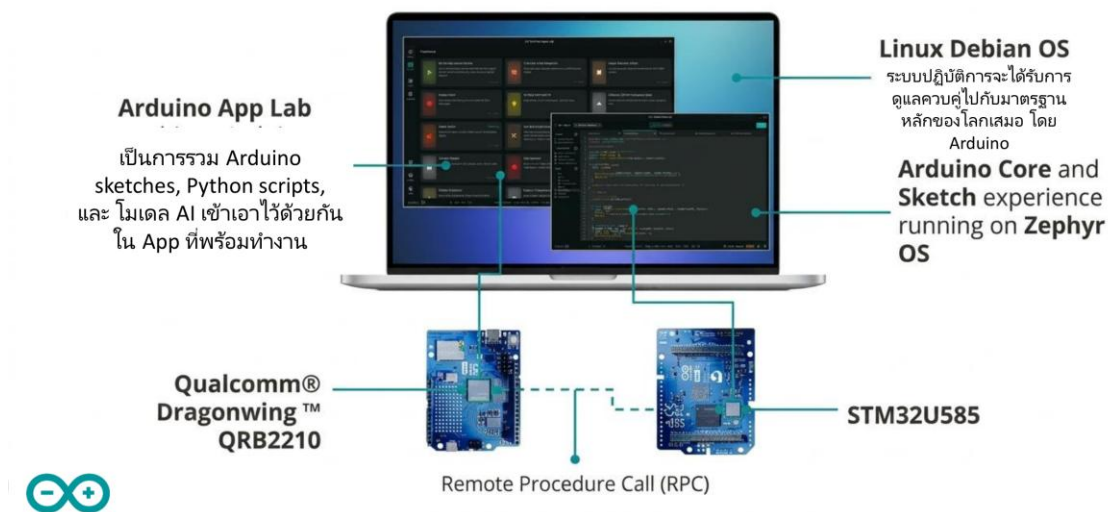
จาก “หลายเดือน” เหลือ “ไม่กี่วัน”



UNO Q รวมการประมวลผล AI การแสดงผล HMI และการควบคุมแบบเรียลไทม์ไว้ในที่เดียว ลดความซับซ้อนของฮาร์ดแวร์และรวมสภาพแวดล้อมการพัฒนาให้เป็นหนึ่งเดียวกัน

สไลด์หน้า 6: จากระบบหลายกล่องที่ซับซ้อน สู่ Arduino UNO Q ที่รวม AI Processing, HMI และ Real-Time Control

สถาปัตยกรรมสองสมอง: เมื่อลินุกซ์ผสมผสานเข้ากับ Arduino



สไลด์หน้า 9: สถาปัตยกรรมสองสมอง Linux Debian OS + Arduino Core บน Zephyr และการเชื่อมผ่าน RPC

Mermaid Diagram (สามารถนำโค้ดนี้ไปวางใน Markdown / Mermaid Viewer เพื่อดูเป็นภาพได้)

flowchart LR

Camera[Camera / Sensor] --> MPU[MPU: Linux + Python + AI]

MPU --> UI[Web UI / Dashboard]

MPU --> RPC[RPC / Message Layer]

RPC --> MCU[MCU: Zephyr + Arduino Core]

MCU --> Actuator[Servo / Motor / Relay / LED]

MCU --> Sensor[Real-time Sensor Read]

Sensor --> RPC

Chapter 3: Hardware Technical Breakdown

1. Overview

บทนี้ลงรายละเอียดฮาร์ดแวร์ของ Arduino UNO Q โดยมองจากมุมวิศวกรรม ไม่ใช่แค่จำสเปก ตัวเลขอย่าง CPU, RAM, Storage, Interface และ Power จะมีความหมายเมื่อเราโยงเข้ากับชนิดของโปรเจกต์ที่ต้องทำ

ตัวอย่างเช่น RAM 2GB หรือ 4GB ไม่ได้แปลว่าแค่เก็บข้อมูลได้มากขึ้น แต่มีผลกับการรัน Docker, Web UI, AI model และหลาย process พร้อมกัน ขณะที่ eMMC 16GB หรือ 32GB มีผลกับพื้นที่เก็บ OS, library, model, log และ asset ของแอป

2. Core Concepts

- QRB2210 เป็น MPU แบบ Quad-core Cortex-A53 สำหรับงาน Linux และ AI/application
- STM32U585 เป็น MCU Cortex-M33 สำหรับงาน control ที่ต้องตอบสนองเร็ว
- LPDDR4 RAM ช่วยให้รัน Linux และ app ได้ลื่นขึ้น
- eMMC เป็น storage ภายใน ใช้เก็บ OS, package, model และข้อมูล
- USB-C ทำหลายหน้าที่ ทั้ง power, data, host/device และ video output
- Wireless connectivity ทำให้บอร์ดเป็น IoT/Edge device ได้ง่าย

3. Technical Breakdown

MPU Dragonwing QRB2210 มี CPU 4 core Arm Cortex-A53 ที่ความเร็ว 2.0 GHz พร้อม Adreno GPU และ ISP สำหรับรองรับกล้อง ดังนั้นฝั่งนี้เหมาะกับงานที่ต้องใช้ระบบปฏิบัติการและประมวลผลข้อมูลระดับสูง

MCU STM32U585 มี Cortex-M33 สูงสุด 160 MHz พร้อม Flash 2 MB และ SRAM 786 KB ตัวเลขนี้อาจดูเล็กเมื่อเทียบกับ RAM ของฝั่ง Linux แต่สำหรับงาน firmware real-time ถือว่าเพียงพอกับการควบคุม sensor/actuator หลายชนิด

ด้านพลังงาน USB-C รับ 5V สูงสุด 3A และ VIN รับ 7-24V ทำให้เหมาะกับทั้งโต๊ะทดลองและระบบฝังตัวที่มีแหล่งจ่ายภายนอก อย่างไรก็ตามเมื่อใช้กล้อง จอ โมดูล wireless และ motor ควรคำนวณ power budget เสมอ

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
------	----------	-----------

Term	ความหมาย	ใช้ทำอะไร
Cortex-A53	CPU core สำหรับ application processor	รัน Linux, Python, AI, web service
Cortex-M33	CPU core สำหรับ microcontroller	ควบคุม real-time task และ low-power embedded
LPDDR4	RAM ความเร็วสูงสำหรับระบบปฏิบัติการ	รองรับ app, Docker, model, buffer
eMMC	หน่วยเก็บข้อมูลแบบฝังบนบอร์ด	เก็บ OS, code, model, logs
MIPI CSI/DSI	interface สำหรับกล้องและจอ	ใช้กับ vision และ display ความเร็วสูง
Qwiic	คอนเนกเตอร์ I2C แบบต่อโมดูลได้ง่าย	ต่อ sensor/module โดยลดการต่อสายผิด

5. Architecture / Workflow

- เริ่มจากเลือกแหล่งจ่ายไฟที่เพียงพอ
- เลือก sensor/camera/display ตาม interface ที่บอร์ดรองรับ
- กำหนดว่างานใดรันบน Linux และงานใดรันบน MCU
- ตรวจสอบหน่วยความจำที่ใช้โดย AI model และ Docker container
- ทดสอบ I/O และ latency ก่อนรวมระบบเต็ม
- ตรวจสอบความร้อนและความเสถียรเมื่อรันต่อเนื่อง

6. Real-world Example

Local Weather Station จากสไลด์เป็นตัวอย่างที่ดี: sensor อ่านค่าความชื้น อุณหภูมิ หรือคุณภาพอากาศ แล้วส่งข้อมูลให้บอร์ดแสดงบน dashboard ฝั่ง Linux สามารถสร้าง Web UI และบันทึกข้อมูล ส่วน MCU อ่าน sensor แบบสμάเสมอและส่งค่าที่นำเชื่อถือเข้าระบบ

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
สเปกแรงกว่าบอร์ด Arduino แบบดั้งเดิมมาก	การใช้ resource มากเกินไปทำให้ระบบช้า
รองรับกล้อง จอ เสียง wireless และ interface หลากหลาย	การต่ออุปกรณ์หลายชิ้นต้องระวัง power/current
มีขนาดมาตรฐาน Arduino UNO ทำให้ต่อยอด shield	interface ความเร็วสูงอย่าง MIPI ต้องใช้

ข้อดี	ข้อควรระวัง / ข้อจำกัด
ได้ง่าย	carrier/สาย/โมดูลที่เข้ากัน

8. Common Mistakes

- ดูแต่ CPU speed แล้วไม่ดู RAM/Storage/Power
- ต่อ motor จากไฟบอร์ดโดยไม่แยก power
- ไม่คิดเรื่อง bandwidth ของกล้องหรือจอ
- ใช้ Docker หลายตัวโดยไม่ดู RAM และ disk

9. Best Practices

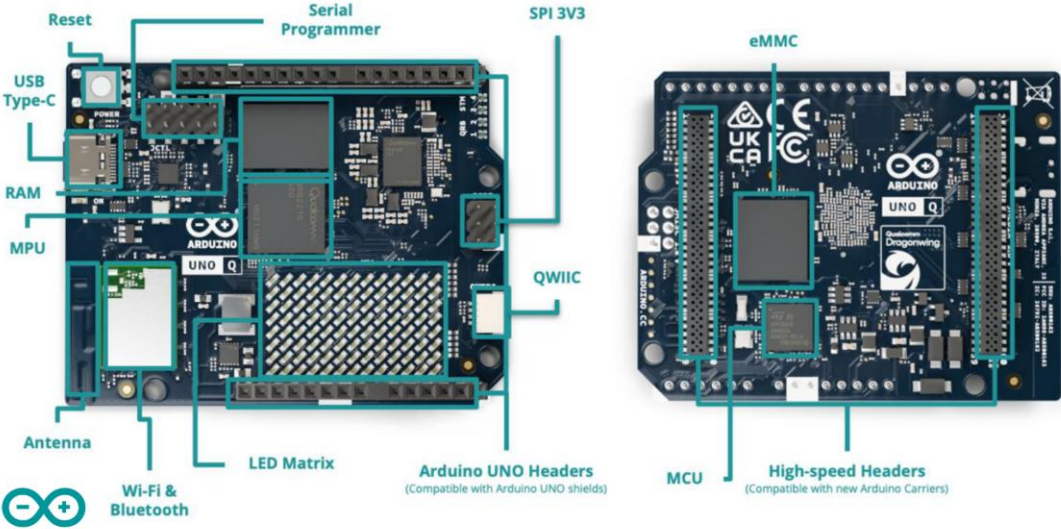
- ทำตาราง resource budget ก่อนเริ่มโปรเจกต์
- ใช้ power supply คุณภาพดีและแยก power สำหรับ actuator
- monitor CPU/RAM/storage/log ระหว่างทดสอบ
- เลือกรุ่น 4GB หากใช้ ROS 2, vision, Docker หรือ AI หลาย component

10. Summary

สรุปท้ายบท

สเปกฮาร์ดแวร์ไม่ได้มีไว้จำ แต่มีไว้ใช้ตัดสินใจออกแบบระบบว่าจะไรควรรันที่ไหน ใช้ memory เท่าไร ต้องการ power เท่าไร และจะรับมือ latency/performance อย่างไร

รายละเอียดทางเทคนิค



สไลด์หน้า 10: ตำแหน่งองค์ประกอบบนบอร์ด เช่น USB-C, RAM, MPU, Qwiic, eMMC, MCU และ headers

รายละเอียดทางเทคนิค

ฮาร์ดแวร์			
รีโมโครโปรเซสเซอร์ (MPU)	Dragonwing QRB2210: • Quad-core Arm® Cortex®-A53 @ 2.0 GHz • ชิปกราฟิก Qualcomm® Adreno™ GPU 3D • 2x ISP (รองรับกล้อง 13 MP + 13 MP หรือ 25 MP) @ 30 fps	ไมโครคอนโทรเลอร์ (MCU)	STM32U585: • Arm® Cortex®-M33 ความเร็วสูงสุด 160 MHz • หน่วยความจำ Flash 2 MB / SRAM 786 KB
หน่วยความจำ (RAM)	2GB หรือ 4GB LPDDR4 (ขึ้นอยู่กับรุ่น)	แหล่งจ่ายไฟ (Power)	• ผ่าน USB-C: 5 VDC สูงสุด 3 A • แรงดันขาเข้า (VIN): 7-24 VDC
พื้นที่เก็บข้อมูล (Storage)	16GB หรือ 32GB eMMC (ขึ้นอยู่กับรุ่น)	พอร์ต USB	1x USB-C (รองรับการสลับโหมด Host/Device, การสลับบทบาทพลังงาน และเอาต์พุตวิดีโอ)
การเชื่อมต่อ (Connectivity)	Wi-Fi 5 (2.4/5GHz) และ Bluetooth 5.1 พร้อมเสาอากาศในตัว	อินเทอร์เฟซ (Interfaces)	I ² C/I ² S, SPI, PWM, CAN, UART, PSSI, GPIO, JTAG, ADC
กล้อง (Camera)	รองรับกล้องแบบ USB และมีพิน 2x MIPI CSI บนคอนเนคเตอร์ JMEDIA	คุณสมบัติเพิ่มเติม (Extra)	• ไฟ LED สี RGB 4 ดวง (ผู้ใช้ควบคุมได้) • หน้าจอ LED Matrix ที่พิกัดขนาด 8x13 • 1x คอนเนคเตอร์ Qwiic (แรงดัน 3V3, I2C) • 1x ปุ่มกดสำหรับผู้ใช้งาน (User push-button) • JCTL: คอนเนคเตอร์สำหรับเก็บนัก MPU ระยะไกล
วิดีโอ (Video)	รองรับการส่งสัญญาณวิดีโอผ่าน USB-C และมีพิน MIPI DSI บนคอนเนคเตอร์ JMEDIA	ขนาด (Dimensions)	68.85 มม. x 53.34 มม. (ขนาดมาตรฐานเท่ากับบอร์ด Arduino UNO)
ระบบเสียง (Audio)	ช่องเสียบไมโครโฟน (IN), หูฟัง (OUT) และ Line OUT บนคอนเนคเตอร์ JMISC		
ซอฟต์แวร์			
ระบบปฏิบัติการ MPU	Linux Debian OS หรือการสนับสนุนการอัปเดตอย่างต่อเนื่อง (upstream support)	การทำคอนเทนเนอร์ (Containerization)	รองรับ Docker และ Docker Compose
ระบบปฏิบัติการแบบ Real-time (RTOS)	Arduino Core บน Zephyr OS	ระบบปฏิบัติการที่รองรับสำหรับ Arduino App Lab	Windows: Windows 10 หรือใหม่กว่า (64-bit) macOS: macOS 11 หรือใหม่กว่า (64-bit) Linux: Ubuntu 22.04 หรือใหม่กว่า และ Debian Trixie (64-bit)

สไลด์หน้า 11: รายละเอียดทางเทคนิคทั้งฮาร์ดแวร์และซอฟต์แวร์ของ Arduino UNO Q

หัวข้อ	Arduino UNO แบบดั้งเดิม	Arduino UNO Q
แนวคิดหลัก	MCU board สำหรับควบคุม I/O และเรียนรู้ embedded	Hybrid board สำหรับ Linux + AI + Real-time control
ระบบปฏิบัติการ	ไม่มี OS เติมรูปแบบ รัน firmware โดยตรง	Linux Debian บน MPU + Zephyr/Arduino Core บน MCU

หัวข้อ	Arduino UNO แบบดั้งเดิม	Arduino UNO Q
งานที่เหมาะสม	Sensor, LED, motor, simple automation	Edge AI, dashboard, robotics, vision, IoT gateway
ภาษาและเครื่องมือ	Arduino C/C++ Sketch	Arduino Sketch, Python, App Lab, Docker, Web UI
ข้อจำกัดหลัก	ประมวลผลหนักและ UI ยาก	ซับซ้อนกว่า ต้องเข้าใจสองโลกและ resource management

Chapter 4: Expansion System: Qwiic, Modulino, Daisy-Chaining และ Carrier Board

1. Overview

ความเก่งของบอร์ด embedded ไม่ได้อยู่ที่ชิปอย่างเดียว แต่อยู่ที่การต่อขยายได้ง่าย ปลอดภัย และไม่ทำให้การทดลองกลายเป็นการไล่สายผิด Arduino UNO Q มีแนวคิดการต่อขยายหลายระดับ ตั้งแต่ Qwiic, Modulino, high-speed connectors ไปจนถึง carrier boards

สำหรับผู้เริ่มต้น Qwiic และ Modulino ช่วยให้ต่อ sensor/module ได้โดยใช้สายมาตรฐาน ลดปัญหาขาคัด polarity หรือ wiring หลวม ส่วนงานขั้นสูงอย่างกล้อง MIPI-CSI, จอ MIPI-DSI และ audio จำเป็นต้องพึ่ง carrier board ที่ออกแบบมารองรับสัญญาณความเร็วสูง

2. Core Concepts

- Qwiic ใช้แนวคิด I2C ต่อโมดูลได้ง่าย
- Daisy-Chaining คือการต่อโมดูลเป็นสายพ่วงเหมือนขบวนรถไฟ
- Carrier Board ขยายพอร์ตเฉพาะทางที่บอร์ดหลักไม่มีพื้นที่ให้วางครบ
- JMEDIA/JMISC เป็นคอนเนกเตอร์สำหรับ media, camera, display, audio และ expansion
- การต่อโมดูลหลายตัวต้องสนใจ address, bandwidth และ power

3. Technical Breakdown

Daisy-Chaining เหมาะกับระบบ sensor หลายตัวที่ใช้ bus เดียวกัน เช่น I2C โดยแต่ละโมดูลมี address ของตัวเอง การต่อแบบนี้ทำให้สายสะอาดและจัดระบบง่าย แต่ถ้า address ขนกันหรือสายยาวเกินไป ระบบจะอ่านค่าไม่เสถียร

Carrier Board มีบทบาทเหมือนฐานขยาย มันไม่ใช่แค่แปลงหัวต่อ แต่ช่วยจัด routing สำหรับสัญญาณความเร็วสูง เช่น camera/display/audio ซึ่งต้องการสัญญาณสะอาด ความยาวเส้นทางเหมาะสม และ connector ที่เข้ากัน

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
Qwiic	คอนเนกเตอร์ I2C แบบเสียบง่าย	ต่อ sensor/module โดยไม่ต้องบัดกรี
Modulino	โมดูลเสริมใน ecosystem Arduino	เพิ่มความสามารถ เช่น sensor, input, output
Daisy-Chaining	ต่ออุปกรณ์เรียงต่อกันบน bus	ลดสายและต่อหลายโมดูลได้ง่าย
Carrier Board	บอร์ดฐานสำหรับขยาย connector	ต่อกล้อง จอ เสียง และอุปกรณ์เสริม
I2C Address	เลขประจำอุปกรณ์บน I2C bus	ป้องกันการชนกันของโมดูลหลายตัว

5. Architecture / Workflow

- เลือก sensor หรือ module ที่ต้องใช้
- ดูว่าใช้ Qwiic/I2C หรือ interface อื่น
- ตรวจสอบ address ของอุปกรณ์แต่ละตัว
- ต่อสายแบบ daisy-chain ให้สั้นและแน่น
- เขียนโปรแกรม scan bus เพื่อตรวจพบอุปกรณ์
- ทดสอบอ่านค่าทีละตัวก่อนรวมหลายตัว
- หากเป็นกล้อง/จอ/audio ให้ใช้ carrier board ที่รองรับ

6. Real-world Example

ระบบ weather station อาจต่อ sensor อุณหภูมิ ความชื้น แสง และคุณภาพอากาศผ่าน Qwiic โดย daisy-chain เป็นชุดเดียว จากนั้นบอร์ดรวบรวมข้อมูล แสดง dashboard และอาจส่ง alert เมื่อค่าผิดปกติ

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
ลด wiring error และทำ prototype เร็ว	I2C มีข้อจำกัดเรื่องความยาวสายและ address
ขยาย sensor ได้หลายตัว	โมดูลมากเกินไปอาจกินไฟหรือทำให้ bus ช้า
carrier ช่วยรองรับงาน media ความเร็วสูง	carrier เฉพาะทางเพิ่มต้นทุนและต้องเลือกให้ตรงรุ่น

8. Common Mistakes

- เสียบโมดูลหลายตัวแล้ว address ซกัน
- ต่อสายยาวมากจน signal ไม่เสถียร
- ลืมคำนวณกระแสรวมของโมดูล
- ใช้สายหรือ carrier ไม่รองรับสัญญาณความเร็วสูง

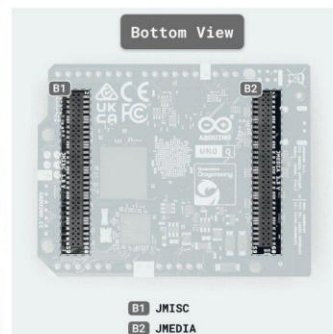
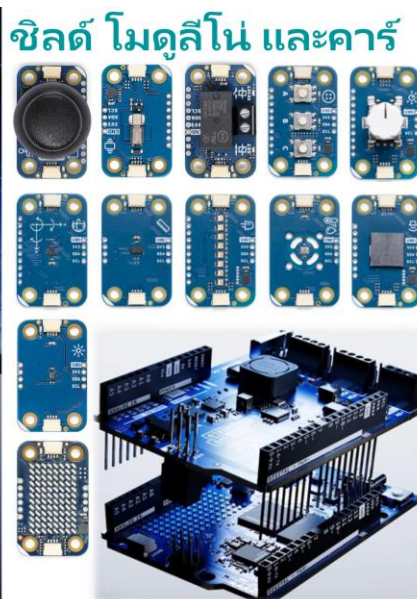
9. Best Practices

- เริ่มจาก bus scan และอ่านค่า sensor ทีละตัว
- ทำผังการต่อสายและ address table
- แยก power สำหรับ module ที่กินไฟมาก
- ใช้สายสั้นและล็อก connector ให้แน่น
- บันทึกเวอร์ชันของ module/carrier ที่ใช้งาน

10. Summary

สรุปท้ายบท

ระบบต่อขยายคือส่วนที่ทำให้ Arduino UNO Q กลายเป็นแพลตฟอร์มจริง ไม่ใช่แค่บอร์ดเดียว การต่อขยายที่ดีช่วยให้ prototype เร็วขึ้น debug ง่ายขึ้น และย้ายจากไอเดียไปสู่ระบบจริงได้มั่นคงกว่า



ข้อต่อความเร็วสูงด้านล่างแบบใหม่ ถูกออกแบบมาเพื่อใช้งานร่วมกับบอร์ดคาร์เรียร์ (carrier board) ซึ่งช่วยให้สามารถเชื่อมต่อกับอุปกรณ์ต่อพ่วงได้หลากหลาย เช่น กล้องแบบ MIPI-CSI, จอแสดงผล MIPI-DSI, ระบบเสียงแบบแอนะล็อก และอุปกรณ์เสริมอื่นๆ อีกมากมาย

สไลด์หน้า 13: Modulino และ carrier board สำหรับกล้อง จอ เสียง และอุปกรณ์เสริม

การเชื่อมต่อ
เป็นขบวน
รถไฟแบบนี้เรา
จะเรียกว่า
“Daisy-
Chaining”



สไลด์หน้า 14: Daisy-Chaining การต่อโมดูลเรียงกันแบบขบวนรถไฟ

JMEDIA and JMISC connectors



Arduino UNO
Media Carrier



Arduino UNO
Breakout Carrier



สไลด์หน้า 15: JMEDIA / JMISC connectors และตัวอย่าง Media Carrier / Breakout Carrier

Chapter 5: Arduino App Lab, Bricks, Python, Sketch และ AI Models

1. Overview

Arduino App Lab คือสภาพแวดล้อมการพัฒนาที่พยายามรวมโลกของ Arduino Sketch, Python script, AI model และ UI เข้าด้วยกัน ทำให้ผู้พัฒนาไม่ต้องกระโดดไปมาระหว่างเครื่องมือจำนวนมากเกินไป

ในสไลด์อธิบาย App Lab ด้วยคำสำคัญ 4 กลุ่ม ได้แก่ สำรวจตัวอย่าง เรียนรู้ สร้าง และแบ่งปัน ซึ่งสะท้อนว่า Arduino ต้องการให้การพัฒนา AI/embedded ไม่ใช่เรื่องของ expert อย่างเดียว แต่เป็น workflow ที่เริ่มจาก example แล้วค่อยปรับแต่งจนกลายเป็น project จริง

2. Core Concepts

- App Lab เป็นศูนย์กลางการพัฒนาและ deploy
- Bricks คือ building blocks ที่รวม Python, Sketch, AI model และ UI
- Ready-to-run apps ใช้เป็นจุดเริ่มต้นในการเรียนและทดสอบไอเดีย
- AI Models สามารถรันบนบอร์ดโดยตรง เช่น image classification, sound recognition
- การ deploy ในคลิกเดียวช่วยลด friction สำหรับผู้เริ่มต้น

3. Technical Breakdown

Bricks ทำหน้าที่คล้ายโมดูลสำเร็จรูป ผู้พัฒนาสามารถเอา Brick หลายตัวมาต่อกัน เช่น web_ui + image_classifier + Arduino Sketch เพื่อสร้างแอปที่ทั้งมองเห็น วิเคราะห์ และควบคุม hardware

การมี Docker และ Docker Compose ช่วยให้จัดการ environment ซับซ้อนได้ดีขึ้น เช่น dependency ของ Python, API service, web backend หรือ AI runtime แต่ก็ต้องแลกกับการใช้ RAM/storage มากขึ้น จึงต้องออกแบบ resource ให้เหมาะสม

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
Arduino App Lab	เครื่องมือรวมสำหรับพัฒนา app บน UNO Q	สร้าง ทดลอง deploy และแชร์โปรเจกต์
Brick	โมดูลซอฟต์แวร์สำเร็จรูป	ประกอบ Python, Sketch, AI และ UI เข้าด้วยกัน
Sketch	โค้ด Arduino C/C++	ควบคุม MCU และฮาร์ดแวร์
Python Script	โค้ด Python บน Linux	ประมวลผลข้อมูล AI web และ automation
On-device AI	AI ที่รันบนบอร์ดโดยตรง	ลด latency และไม่ต้องส่งข้อมูลขึ้น cloud
Docker Compose	เครื่องมือจัดการหลาย container	รัน service หลายตัวให้ reproducible

5. Architecture / Workflow

25. เลือก example ที่ใกล้เคียงกับไอเดีย
26. ดูว่า example ใช้ Bricks อะไรบ้าง
27. แก้ไข Python script หรือ Sketch ตาม sensor/hardware จริง
28. เพิ่ม AI model เช่น image classifier หรือ sound recognizer
29. สร้าง Web UI หรือ dashboard เพื่อแสดงผล
30. ทดสอบ local แล้ว deploy ไปยังบอร์ด
31. บันทึก version dependency และ configuration

6. Real-world Example

โปรเจกต์ตรวจจับบุคคคล: ใช้ Brick สำหรับ camera input, person classification model, web dashboard และ Arduino Sketch ที่ควบคุม LED/relay เมื่อพบคนในพื้นที่ที่กำหนด ระบบนี้สามารถใช้ใน security prototype หรือ smart building ได้

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
ลดเวลา setup environment	อาจซ่อนรายละเอียดบางอย่างจนมือใหม่ไม่เข้าใจ layer จริง
เริ่มจากตัวอย่างได้เร็ว	project ใหญ่ยังต้องมีการจัดการ dependency และ version
รวม AI + UI + hardware ใน workflow เดียว	Docker ใช้ resource สูงกว่าสคริปต์ธรรมดา

8. Common Mistakes

- ลาก Brick มาต่อโดยไม่เข้าใจข้อมูลที่ไหลระหว่างกัน
- deploy โดยไม่ดู log เมื่อ error
- ใช้ model ใหญ่เกินความจำ
- ไม่แยก configuration ออกจาก code

9. Best Practices

- อ่าน structure ของ example ก่อนแก้
- ตั้งชื่อ Brick และ service ให้สื่อความหมาย

- เก็บ configuration เช่น API key, threshold, camera index แยกเป็นไฟล์
- ใช้ logging ที่ชัดเจนทั้ง Python และ Sketch
- ทดสอบ model ด้วยข้อมูลจริงก่อนนำไปควบคุม actuator

10. Summary

สรุปท้ายบท

App Lab และ Bricks คือสะพานที่ทำให้ Arduino UNO Q เข้าใกล้แนวคิด “สร้างแอป embedded AI” มากกว่าการเขียน firmware อย่างเดียว ผู้ใช้จึงควรเข้าใจทั้งความสะอาดของเครื่องมือและ layer ที่อยู่เบื้องหลัง



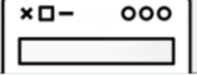
สไลด์หน้า 22: Arduino App Lab

เสริมพลังให้นักพัฒนาด้วย Arduino App Lab



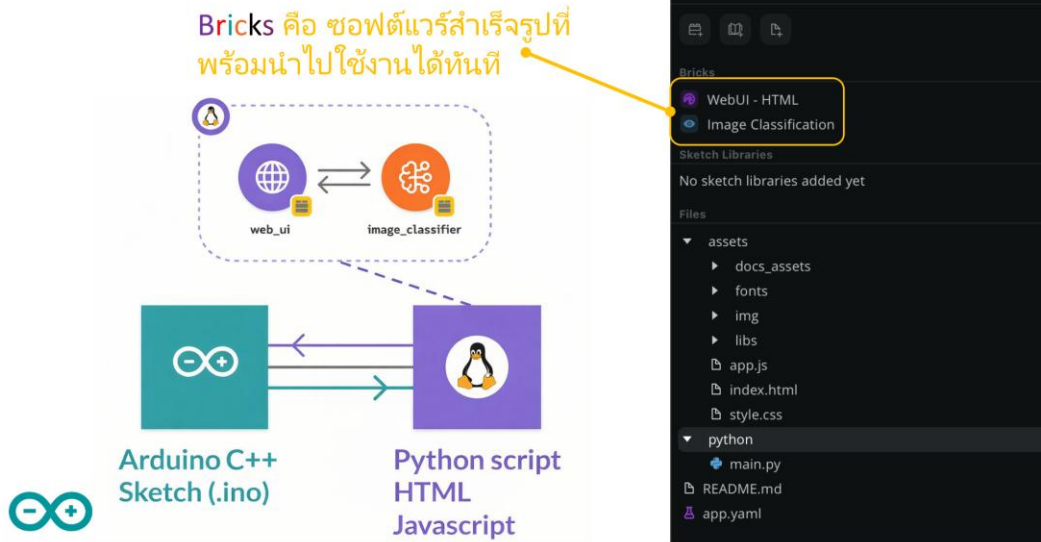
สไลด์หน้า 23: App Lab ช่วยสำรวจตัวอย่าง เรียนรู้ สร้าง และแบ่งปัน

องค์ประกอบพื้นฐานของ Arduino App Lab

 ตัวอย่าง แอปที่พร้อมใช้งานทันที (Ready-to-run apps) ที่แสดงให้เห็นว่า Bricks, AI Models, Scripts และ Sketches ทำงานร่วมกันอย่างไร ใช้ตัวอย่างเหล่านี้เพื่อ - เรียนรู้ (learn) - ทดสอบไอเดีย (test ideas) - หรือเริ่มต้นโปรเจกต์ของคุณได้อย่างรวดเร็ว (jumpstart your project) ตัวอย่างทุกตัวสามารถปรับแต่งได้ทั้งหมด	 Bricks Bricks คือ โมดูลแบบบล็อก ที่ช่วยให้คุณสามารถรวม - Python scripts - Arduino Sketches - AI models เข้าด้วยกัน เพื่อจัดการเซ็นเซอร์ ความคมชัดแว้วร์ และสร้าง อินเทอร์เฟซแบบกำหนดเองได้ ทั้งหมดนี้ทำได้ด้วยความง่ายแบบลากแล้ววาง (drag-and-drop)	 โมเดล AI เพิ่มความสามารถด้าน ปัญญาประดิษฐ์ (AI) ให้กับแอปของคุณ เช่น - การจำแนกภาพ (image classification) - การจดจำเสียง (sound recognition) โดยทำงาน โดยตรงบนบอร์ด (on-device) ไม่จำเป็นต้องใช้คลาวด์ เพียงแค่ลากโมเดลไปใส่ใน Brick ก็ใช้งานได้	 แอปที่คุณสร้างเอง ผลงานที่คุณสร้างขึ้นเอง โดยสามารถ - แก้ไขตัวอย่าง (modifying examples) - หรือประกอบ Bricks ใหม่ตั้งแต่เริ่มต้น (assembling Bricks from scratch) คุณสามารถปรับแต่ง - ตรรกะการทำงาน (logic) - ส่วนติดต่อผู้ใช้ (UI) - พฤติกรรมของแอป (behavior) ให้ตรงกับแนวคิดของคุณ แล้ว Deploy ได้ในคลิกเดียว
--	---	--	---

สไลด์หน้า 24: องค์ประกอบพื้นฐาน ได้แก่ Examples, Bricks, AI Models และแอปที่สร้างขึ้นเอง

Arduino App Lab: The Bricks



สไลด์หน้า 25: ตัวอย่าง Bricks ที่รวม Web UI, Image Classification, Arduino Sketch, Python, HTML และ JavaScript

Chapter 6: AI / Embedded Workflow: Sensor Flow, Vision Flow และ Real-time Processing

1. Overview

เมื่อรวม AI กับ embedded system เราไม่ควรคิดแค่ “โมเดลแม่นยำ” แต่ต้องคิดทั้ง pipeline ตั้งแต่รับข้อมูล เตรียมข้อมูล รัน inference ตัดสินใจ ส่งคำสั่ง และควบคุม hardware ความแม่นยำของโมเดลเป็นเพียงส่วนหนึ่งของระบบทั้งหมด

Arduino UNO Q เหมาะกับ pipeline ที่ข้อมูลเกิดที่ขอบเครือข่าย เช่น กล้อง เซนเซอร์ เสียง หรือการเคลื่อนไหว แล้วบอร์ดประมวลผลในพื้นที่ทันทีโดยไม่ต้องส่งข้อมูลดิบทั้งหมดไป cloud

2. Core Concepts

- Sensor Flow คือเส้นทางข้อมูลจาก sensor ไปสู่ decision และ actuator
- Vision Flow คือเส้นทางจาก camera capture ไปสู่ image preprocessing, inference และ response
- Real-time Processing คือการควบคุมงานที่ต้องตอบสนองภายในเวลาที่กำหนด
- Latency Budget คือการแบ่งเวลาสูงสุดให้แต่ละขั้น เช่น capture 30 ms, inference 100 ms, control 10 ms
- Feedback Loop คือระบบที่อ่านผลลัพธ์แล้วปรับการควบคุมต่อเนื่อง

3. Technical Breakdown

Pipeline ที่ดีต้องแยกแยะระหว่าง data plane และ control plane data plane คือข้อมูลจำนวนมาก เช่น image frame หรือ sensor stream ส่วน control plane คือคำสั่งเล็ก ๆ เช่น class_id, confidence, motor_angle, alarm_on การส่งเฉพาะผลลัพธ์ที่จำเป็นไปยัง MCU จะทำให้ระบบเบาและตอบสนองดีขึ้น

ฝั่ง AI ควรทำ preprocessing ให้เหมาะสม เช่น resize ภาพ ลด noise normalize หรือ sample เสียงเป็นช่วง ๆ แล้วค่อยส่งเข้าโมเดล เมื่อได้ผลลัพธ์ต้องมี threshold และ rule เพื่อป้องกัน false positive ที่อาจทำให้ actuator ทำงานผิด

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
Inference	การให้โมเดล AI ทำนายผลจากข้อมูลใหม่	ใช้จำแนกภาพ เสียง คน วัตถุ หรือความผิดปกติ
Preprocessing	การเตรียมข้อมูลก่อนเข้าโมเดล	resize, normalize, filter, crop, encode
Latency	เวลาหน่วงตั้งแต่รับข้อมูลจนตอบสนอง	ใช้วัดความเร็วและความเหมาะสมกับ real-time
Threshold	ค่าขีดจำกัดในการตัดสินใจ	ลดการตอบสนองผิดจากความมั่นใจต่ำ
Feedback Control	ควบคุมโดยดูผลตอบกลับ	ใช้ใน motor, robot arm, closed-loop system

5. Architecture / Workflow

- 32. รับข้อมูลจาก sensor/camera/microphone
- 33. ทำ preprocessing ให้ข้อมูลอยู่ในรูปแบบที่โมเดลต้องการ
- 34. รัน AI inference บน MPU
- 35. ตรวจสอบ confidence และ rule เฉพาะระบบ
- 36. ส่ง command ขนาดเล็กไป MCU
- 37. MCU ควบคุม actuator และอ่าน feedback
- 38. ส่ง status กลับไป dashboard/log

6. Real-world Example

ระบบตรวจจับความผิดปกติในเครื่องจักร: microphone หรือ vibration sensor รับสัญญาณ ฝั่ง Linux วิเคราะห์ด้วย anomaly detection หากพบรูปแบบผิดปกติจะส่งคำสั่งให้ MCU เปิดไฟเตือน หยุด motor หรือส่ง notification โดยระบบต้องออกแบบไม่ให้ false alarm บ่อยจนผู้ใช้ไม่เชื่อถือ

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
ตอบสนองเร็วเพราะประมวลผลที่ edge	ต้อง optimize model ให้เหมาะกับ hardware
ลดข้อมูลที่ต้องส่งผ่าน network	real-time control ต้องแยกจาก AI ที่ timing ไม่นิ่ง
ปกป้อง privacy โดยไม่ส่งภาพ/เสียงดิบออกนอกระบบ	ข้อมูลจริงอาจต่างจาก dataset ที่ใช้ฝึก

8. Common Mistakes

- วัดเฉพาะ accuracy แต่ไม่วัด latency
- ใช้ confidence threshold ต่ำเกินไป
- ให้ AI สั่ง actuator โดยไม่มี safety rule
- ไม่บันทึก log ของ input/output ทำให้ debug ยาก

9. Best Practices

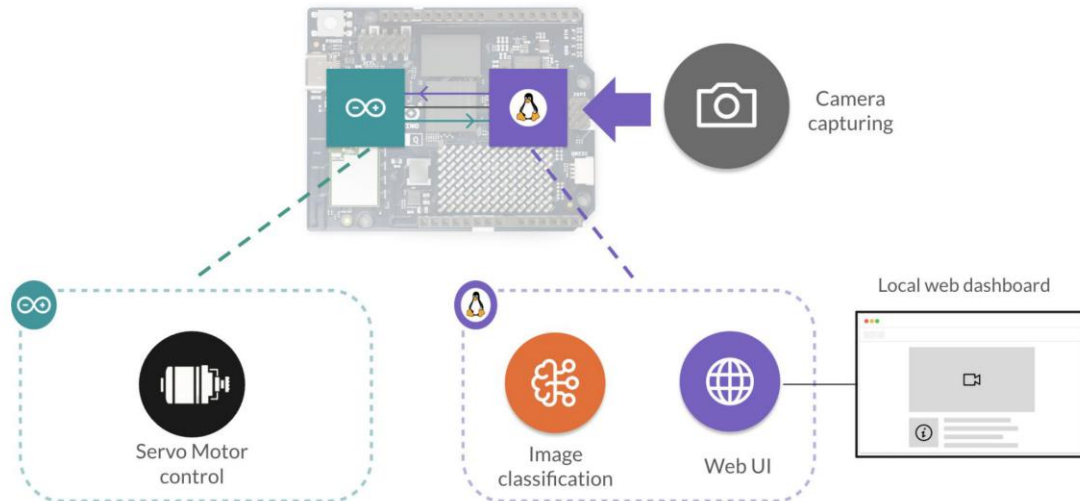
- ออกแบบ pipeline ก่อนเขียน code
- ใช้ model ที่เบาและเหมาะกับ edge
- ทำ fallback mode เมื่อ AI ไม่มั่นใจ
- บันทึก sample ที่ทำนายผิดเพื่อนำไปปรับปรุง model
- แยก emergency stop หรือ safety control ไว้ที่ MCU

10. Summary

สรุปท้ายบท

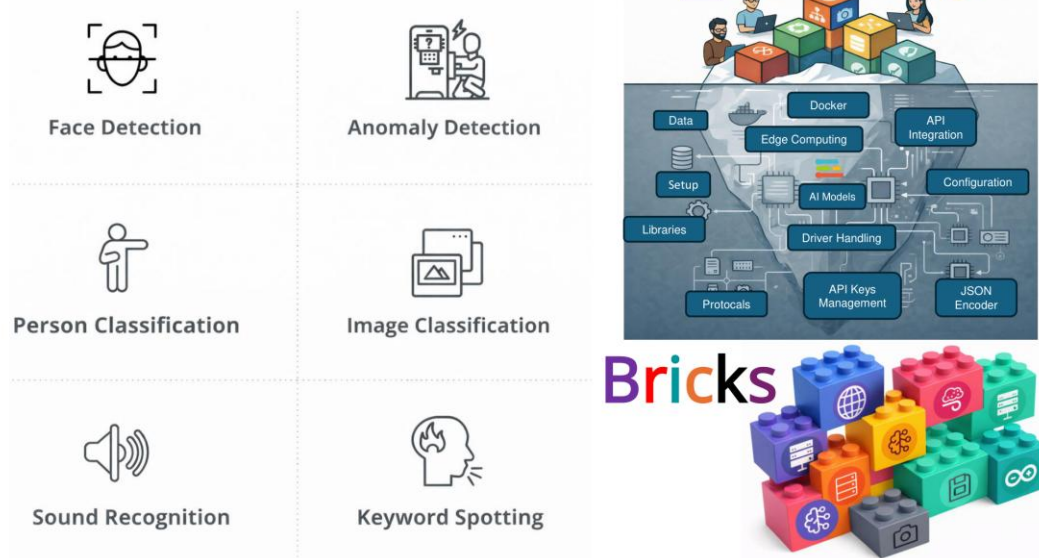
AI/Embedded ที่ต้องคิดเป็นระบบครบวงจร ไม่ใช่แค่โหลดโมเดลมารัน สิ่งที่ทำให้ระบบใช้งานจริงได้คือ pipeline ที่มี latency ชัดเจน มี safety rule และแยกงาน control ไปยังส่วนที่เหมาะสม

สถาปัตยกรรมระหว่างไมโครโปรเซสเซอร์และ



สไลด์หน้า 12: ตัวอย่างสถาปัตยกรรม camera capture, image classification, Web UI และ servo motor control

โมเดล AI ทรงพลัง พร้อมนำไปใช้งานได้ทันที



สไลด์หน้า 26: โมเดล AI พร้อมใช้งาน เช่น Face Detection, Anomaly Detection, Person Classification, Image Classification, Sound Recognition และ Keyword Spotting

Mermaid Diagram (สามารถนำโค้ดนี้ไปวางใน Markdown / Mermaid Viewer เพื่อดูเป็นภาพได้)

```
sequenceDiagram
```

```
participant S as Sensor/Camera
```

```
participant L as Linux MPU
```

participant A as AI Model

participant M as MCU/RTOS

participant O as Actuator

S->>L: raw data / frame / audio

L->>L: preprocessing

L->>A: inference request

A->>L: label + confidence

L->>L: threshold + business rule

L->>M: command/status via RPC

M->>O: PWM/GPIO/Control signal

M->>L: feedback + health status

Pipeline Stage	ทำงานบนส่วนใด	เหตุผล
Capture / Buffer	MPU หรืออุปกรณ์ก้อง	ข้อมูลภาพ/เสียงมีขนาดใหญ่และต้องใช้ driver/OS
Preprocessing	MPU	ใช้ library Python/OpenCV ได้สะดวก
AI Inference	MPU	ต้องใช้ CPU/GPU/AI runtime และ memory มากกว่า MCU
Decision Rule	MPU + MCU	MPU วิเคราะห์ เงื่อนไข safety สำคัญควรสะท้อนที่ MCU
Actuator Control	MCU	ต้องการ timing แม่นและ deterministic response
Dashboard / Logging	MPU	เหมาะกับ web, file, database และ network

Chapter 7: Applications & Engineering View

1. Overview

สไลด์ยกตัวอย่างการใช้งานหลายกลุ่ม เช่น Robotics, Home & Building Automation และ Prototyping

จุดร่วมของงานเหล่านี้คือไม่ได้ต้องการแค่ sensor หรือ actuator แต่ต้องการการรับรู้ การตัดสินใจ และการควบคุมในระบบเดียว

เมื่อมองแบบวิศวกร เราต้องแปลงไอเดียให้กลายเป็น requirement เช่น latency เท่าไร ใช้พลังงานเท่าไร ต้องทำงาน offline ได้ไหม ต้องเก็บ log ไหม ต้องอัปเดตซอฟต์แวร์อย่างไร และถ้าระบบผิดพลาดต้อง fail-safe อย่างไร

2. Core Concepts

- Robotics ต้องการ perception + planning + control
- Smart Home ต้องการ UI, network, automation และ privacy
- Prototyping ต้องการความเร็วในการทดลองและการต่อขยาย
- Edge computer ขนาดเล็กต้องบาลานซ์ performance, power และ thermal
- ระบบใช้งานจริงต้องมี deployment, monitoring และ maintenance

3. Technical Breakdown

งาน robotics เช่น autonomous delivery bot หรือ gesture-following robot ต้องใช้กล้องหรือ sensor เพื่อรับรู้โลก ฝั่ง Linux วิเคราะห์และวาง logic ส่วน MCU ควบคุม motor/servo ให้ตอบสนองอย่างต่อเนื่อง การแยกหน้าที่นี้ช่วยให้ robot มีทั้งความฉลาดและการเคลื่อนไหวที่นิ่ง

งาน smart home เช่น doorbell จดจำใบหน้า หรือ home assistant ที่ยักกระดักด้วยเสียงและท่าทาง จะได้ประโยชน์จาก on-device AI เพราะข้อมูลส่วนตัว เช่นภาพใบหน้าหรือเสียง ไม่จำเป็นต้องส่งขึ้น cloud เสมอไป

งาน prototyping เช่น smart kiosk หรือ compact edge computer ต้องการให้ developer สร้าง demo ได้เร็ว มีจอ มี web UI มี camera และรัน logic ในเครื่องเดียว

4. Important Terms

Term	ความหมาย	ใช้ทำอะไร
Robotics	ระบบหุ่นยนต์ที่รับรู้และเคลื่อนไหว	ใช้ camera, AI, motor control, ROS 2
HMI	Human-Machine Interface	ใช้ให้มนุษย์ดูสถานะและควบคุมระบบ
Home Assistant	แพลตฟอร์ม automation บ้าน	เชื่อมต่อ sensor/device และสร้าง automation
Prototype	ต้นแบบสำหรับพิสูจน์แนวคิด	ทดสอบ feasibility ก่อนทำ product
Fail-safe	โหมดปลอดภัยเมื่อระบบผิดปกติ	ลดความเสียหายเมื่อ software/hardware ล้มเหลว

5. Architecture / Workflow

39. เริ่มจากปัญหาจริงที่ต้องแก้
40. ระบุ input เช่น camera, voice, sensor หรือ user action
41. ระบุ output เช่น motor, relay, notification, dashboard
42. กำหนด latency และ reliability target
43. ออกแบบ pipeline และแบ่งงาน MPU/MCU
44. สร้าง prototype แบบเล็กที่สุดที่พิสูจน์ได้
45. ทดสอบกับสภาพแวดล้อมจริงและปรับปรุง

6. Real-world Example

แขนกล DIY พร้อม visual feedback: กล้องดูตำแหน่งวัตถุ ฝั่ง MPU วิเคราะห์ตำแหน่งแล้วคำนวณ target ส่วน MCU ควบคุม servo ของแขนกลให้เคลื่อนที่ไปยังตำแหน่งที่ต้องการ ถ้า feedback บอกว่าตำแหน่งยังคลาดเคลื่อน ระบบจะปรับซ้ำจนเข้าใกล้เป้าหมาย

7. Pros / Cons

ข้อดี	ข้อควรระวัง / ข้อจำกัด
เหมาะกับงาน demo และ prototype ขั้นสูง	งานภาคสนามต้องระวัง power, heat, vibration และ network
มี ecosystem ช่วยให้เริ่มเร็ว	ระบบจริงต้องมี enclosure และ protection
รองรับทั้ง AI, UI และ hardware control	prototype ที่ไม่ได้แปลว่า product พร้อมขายทันที

8. Common Mistakes

- เริ่มจาก solution ก่อนเข้าใจ problem
- ไม่กำหนด acceptance criteria
- ทดสอบแต่ในห้อง ไม่ทดสอบแสง เสียง หรือสภาพแวดล้อมจริง
- มองข้าม maintenance และ update

9. Best Practices

- เขียน requirement แบบวัดได้ เช่น latency < 200 ms
- ทำ test case สำหรับ sensor noise และ failure mode
- แยก demo, prototype, pilot และ production ให้ชัด

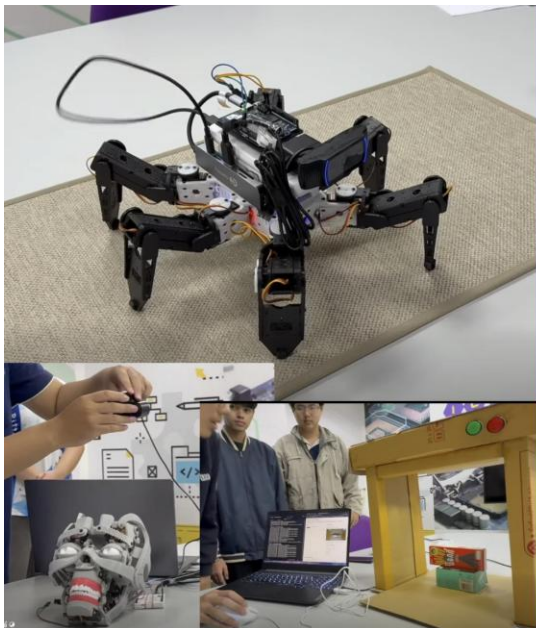
- ทำเอกสาร wiring และ deployment ตั้งแต่ต้น
- คิดเรื่อง power และ enclosure เร็วกว่าที่คิดว่าจำเป็น

10. Summary

สรุปท้ายบท

Arduino UNO Q เปิดโอกาสให้ prototype ที่เคยต้องใช้หลายบอร์ดรวมกันกลายเป็นระบบเดียว

แต่การใช้งานจริงยังต้องคิดแบบวิศวกรเต็มรูปแบบ ทั้ง performance, power, safety, maintenance และ deployment



การประยุกต์ใช้งาน

หุ่นยนต์ (Robotics)

- หุ่นยนต์ส่งของอัตโนมัติ (Autonomous Delivery Bot)
- หุ่นยนต์ที่ติดตามท่าทางของมนุษย์ (Gesture-Following Robot)
- โปรเจกต์ที่ใช้ ROS 2 (แนะนำรุ่น 4GB)
- แขนกล DIY ที่มีระบบป้อนกลับทางภาพ (DIY Robot Arm with Visual Feedback)

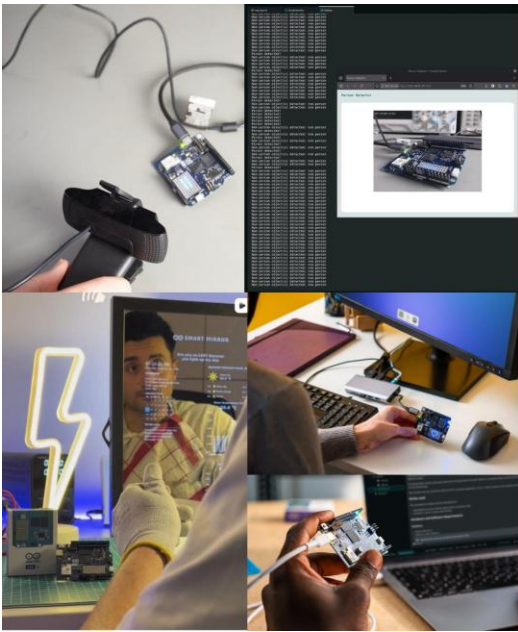
สไลด์หน้า 16: ตัวอย่าง Robotics เช่น autonomous delivery bot, gesture-following robot, ROS 2 และ robot arm

การประยุกต์ใช้งาน

ระบบบ้านและอาคารอัจฉริยะ (Home & Building Automation)

- เครื่องประต้อัจฉริยะแบบโต้ตอบ พร้อมการจดจำใบหน้า
- ระบบควบคุมบ้านด้วยเสียงและท่าทาง
- ยกระดับระบบ Home Assistant ให้ล้ำขึ้นไปอีกขั้น
- ศูนย์ควบคุมสภาพอากาศแบบเฉพาะบุคคล
- ระบบตรวจสอบและวิเคราะห์การใช้พลังงานอัจฉริยะ





การประยุกต์ใช้งาน

การสร้างต้นแบบ (Prototyping)

- ระบบตรวจสอบด้วยภาพ
- คิวอัจฉริยะ
- คอมพิวเตอร์ Edge ขนาดกะทัดรัด

สไลด์หน้า 18: ตัวอย่าง Prototyping เช่น visual inspection, smart kiosk และ compact edge computer

Chapter 8: Deep Technical Analysis, System Design Thinking และ Developer Notes

Deep Technical Analysis

การออกแบบระบบบน Arduino UNO Q ควรมองเป็น layered architecture ไม่ใช้การเขียนโปรแกรมไฟล์เดียวแล้วจบ โดยทั่วไปสามารถแบ่งเป็น Hardware Layer, Real-time Control Layer, Linux Application Layer, AI/Inference Layer, UI/Integration Layer และ Deployment/Monitoring Layer

Layer	หน้าที่	ตัวอย่างบน Arduino UNO Q	สิ่งที่ต้องระวัง
Hardware Layer	เชื่อมต่อ sensor, camera, actuator, power	Qwiic, MIPI CSI, GPIO, PWM, USB-C	ไฟไม่พอ สายหลวม noise และ heat
Real-time Layer	ควบคุม timing และ I/O	STM32U585 + Zephyr + Arduino Core	interrupt, latency, watchdog, safety
Linux Layer	รัน app, service, file, network	Debian OS, Python, Docker	resource, boot time, security update
AI Layer	preprocess และ inference	Image classification, sound recognition	model size, accuracy, latency
UI/Integration	dashboard, API, HMI	Web UI, App Lab, local	auth, UX, network failure

Layer	หน้าที่	ตัวอย่างบน Arduino UNO Q	สิ่งที่ต้องระวัง
		dashboard	
Deployment	ติดตั้ง อัปเดต monitor	Docker Compose, logs, config	version drift, rollback, disk full

System Design Thinking

ระบบนี้ถูกออกแบบมาเพื่อแก้ปัญหาสำคัญของ embedded AI คือการทำงาน AI, UI และ real-time control มักแยกกันอยู่คนละอุปกรณ์ การแยกแบบเดิมทำให้ต้องต่อสายมาก ต้องจัดการ protocol หลายแบบ และ debug ยากเมื่อข้อมูลไหลผิดจังหวะ

ทำไมต้องแยก MPU และ MCU

เพราะ Linux เก่งเรื่องความยืดหยุ่นและประมวลผลหนัก แต่ timing ไม่นิ่งเท่า real-time control ส่วน MCU เก่งเรื่อง timing และ hardware control แต่ไม่เหมาะกับ AI/model/web app ขนาดใหญ่ การแยกหน้าที่ทำให้ระบบใช้จุดแข็งของทั้งสองโลก

ทำไม Linux + RTOS ถึงทำงานร่วมกัน

Linux จัดการ application, network, AI และ UI ส่วน RTOS จัดการ task ที่ต้องตอบสนองตามเวลา เช่น motor control หรือ sensor sampling เมื่อรวมกันจะได้ระบบที่ทั้ง “ฉลาด” และ “ควบคุมได้แม่นยำ”

Learning Expansion

Linux Basics ที่ควรรู้

- Process: โปรแกรมที่กำลังรัน เช่น Python app หรือ web server
- Service: process ที่รันเป็นระบบหลังบ้าน เช่น dashboard หรือ API
- File system: ที่เก็บ code, config, model และ log
- Network: การเชื่อมต่อ Wi-Fi, IP, port และ API
- Package/Container: วิธีติดตั้ง dependency และทำให้ environment ทำซ้ำได้

RTOS Basics ที่ควรรู้

- Task/Thread: งานย่อยที่รันแยกกัน
- Scheduler: ตัวจัดลำดับว่า task ไหนได้ทำงานเมื่อไร
- Interrupt: เหตุการณ์ที่ต้องตอบสนองทันที เช่น sensor trigger

- Determinism: ความสามารถในการคาดเดาเวลาตอบสนอง
- Watchdog: ระบบตรวจจับว่าซอฟต์แวร์ค้างหรือไม่

Embedded AI Basics

- เลือกโมเดลที่เหมาะสมกับ hardware ไม่ใช่โมเดลใหญ่ที่สุด
- ลดขนาด input เช่น resize ภาพเพื่อช่วยลดเวลา inference
- ใช้ threshold และ smoothing เพื่อลดผลทำนายแกว่ง
- เก็บตัวอย่างที่ทำนายผิดเพื่อนำไปปรับปรุง dataset
- วัด latency end-to-end ไม่ใช่วัดเฉพาะ inference time

Practical Engineering View

ประเด็นวิศวกรรม	คำถามที่ต้องตอบ	แนวทางที่ดี
Latency	ตั้งแต่ sensor ถึง actuator ใช้เวลากี่ ms?	แบ่ง latency budget และวัดจริงทุก stage
Performance	CPU/RAM/Storage พอไหม?	ใช้ monitoring และเลือก model/container ให้เหมาะสม
Power	อุปกรณ์รวมกินกระแสเท่าไร?	แยก power actuator และใช้ supply ที่มี headroom
Thermal	ร้อนต่อเนื่องแล้วร้อนไหม?	ทดสอบ load ยาวและออกแบบ airflow/enclosure
Deployment	อัปเดต code/model อย่างไร?	ใช้ versioning, config file, rollback plan
Scaling	ถ้าติดตั้งหลายจุดจะจัดการยังไง?	ทำ image/config มาตรฐานและ centralized logging

Developer Notes

สิ่งที่มีมือใหม่มักไม่เข้าใจ

การมี Linux ไม่ได้แปลว่าควรทำทุกอย่างบน Linux และการมี MCU ไม่ได้แปลว่าต้องยัด logic ทั้งหมดลง firmware
 ให้คิดแยกตามธรรมชาติของงาน: งานหนักและยืดหยุ่นอยู่บน MPU งาน timing/safety อยู่บน MCU

Debugging Tips

- เริ่ม debug จาก layer ล่างขึ้นบน: power -> wiring -> bus scan -> firmware -> RPC -> Python -> AI -> UI
- บันทึก timestamp ทุกชั้นเพื่อหา latency จริง
- แยก log ของ MPU และ MCU แล้วทำ correlation ด้วย request id หรือ timestamp
- เมื่อ actuator ทำงานผิด ให้ดูทั้ง command ที่ส่งและ feedback ที่ได้รับ
- ทดสอบด้วย input ปลอมก่อนต่อ hardware จริง เช่น mock camera frame หรือ fake sensor value

Recommended Tools

- Serial Monitor / console log สำหรับตรวจ firmware และข้อความ debug
- I2C scanner สำหรับเช็คโมดูล Qwiic/Modulino
- htop / top สำหรับดู CPU และ RAM บน Linux
- docker compose logs สำหรับดู service log
- OpenCV หรือ camera test script สำหรับตรวจ vision pipeline
- Git สำหรับ version control และย้อนกลับเมื่อแก้พัง

Development Workflow ที่แนะนำ

46. พิสูจน์ hardware ที่ละชิ้น
47. เขียน firmware MCU ให้ควบคุม output พื้นฐานได้ก่อน
48. เขียน Python/AI pipeline แบบยังไม่ควบคุม actuator
49. เชื่อมสองฝั่งด้วย command ง่าย ๆ
50. เพิ่ม UI/dashboard
51. เพิ่ม safety rule และ fallback
52. ทดสอบ end-to-end ด้วยข้อมูลจริง
53. ทำเอกสาร wiring, dependency, config และขั้นตอน deploy

สรุปภาพรวมของ Arduino UNO Q



สไลด์หน้า 27: สรุปภาพรวม Arduino UNO Q เช่น Dual Brain, Freedom to Build, App Lab, AI-ready และ Connected by Design

Exam Prep: สรุปพร้อมสอบ

What to Remember

- Arduino UNO Q คือบอร์ด hybrid ที่รวม MPU + MCU
- MPU เหมาะกับ Linux, AI, Web UI, Python, Docker
- MCU เหมาะกับ real-time control, sensor sampling, PWM, GPIO
- App Lab และ Bricks ช่วยรวม Sketch, Python, AI model และ UI
- Qwiic/Modulino/Daisy-Chaining ช่วยให้ต่อโมดูลได้ง่าย
- Carrier board ใช้ขยาย media/connector ความเร็วสูง เช่น camera, display, audio
- Edge AI ช่วยลด latency และลดการพึ่งพา cloud

Key Takeaways

คำถามที่พบบ่อย	แนวคำตอบที่ควรจำ
Arduino UNO Q ต่างจาก Arduino เดิมอย่างไร?	เดิมเน้น MCU และ I/O control ส่วน UNO Q รวม Linux MPU + real-time MCU จึงทำ AI, UI และ control ได้ในบอร์ดเดียว
ทำไมต้องมี MPU และ MCU?	MPU ทำงานหนักและยืดหยุ่น เช่น Linux/AI/UI ส่วน MCU ทำงาน timing สำคัญ เช่น PWM/sensor/motor
Edge AI มีข้อดีอะไร?	ลด latency, ลด bandwidth, เพิ่ม privacy และทำงานได้แม้ network

คำถามที่พบบ่อย	แนวคำตอบที่ควรจำ
	ไม่สมบูรณ์
Bricks คืออะไร?	โมดูลซอฟต์แวร์สำเร็จรูปที่รวม Python, Sketch, AI model และ UI เพื่อสร้างแอปได้เร็ว
Daisy-Chaining คืออะไร?	การต่อโมดูลหลายตัวเรียงกันบน bus เดียว เหมือนขบวนรถไฟ โดยต้องระวัง address/power/signal
Real-time สำคัญในงานใด?	งาน motor, servo, robot, safety control และ sensor sampling ที่ต้องตอบสนองตรงเวลา

Mini Quiz

คำถาม	คำตอบ
ถ้าต้องสร้างกล้องตรวจจับวัตถุแล้วสั่ง servo ควรแบ่งงานอย่างไร?	ให้ MPU จับภาพและรัน image classification จากนั้นส่งคำสั่งสั้น ๆ ให้ MCU ควบคุม servo
ถ้า dashboard เข้าแค่ servo ยังต้องตอบสนอง ควรทำอย่างไร?	แยก control loop ของ servo ไว้ที่ MCU และให้ dashboard เป็นเพียง UI/status บน MPU
ทำไมไม่ควรส่งภาพดิบทุก frame ไปยัง MCU?	เพราะภาพมีขนาดใหญ่และ MCU ไม่เหมาะกับประมวลผลภาพ ควรส่งเฉพาะผลลัพธ์ เช่น label/confidence/command
เมื่อใช้โมดูล Qwiic หลายตัวต้องตรวจสอบอะไร?	ตรวจสอบ I2C address, power, ความยาวสาย และความเสถียรของ bus
สำหรับโปรเจกต์ ROS 2 ควรเลือกรุ่นใด?	ควรเลือกรุ่น RAM 4GB เพราะ ROS 2 และ service หลายตัวใช้ memory มาก

Final Learning Summary

หลังอ่านเอกสารนี้ ผู้อ่านควรเข้าใจ Arduino UNO Q ในฐานะแพลตฟอร์ม embedded AI ไม่ใช่แค่บอร์ดทดลองทั่วไป

จุดแข็งของมันคือการรวม Linux application processor กับ microcontroller real-time ทำให้โปรเจกต์ที่ต้องมี AI, UI, sensor และ actuator สามารถอยู่ในระบบเดียวได้

Skills Learned

- อ่านสเปกฮาร์ดแวร์และโยงกับงานจริงได้

- อธิบาย MPU vs MCU และ Linux vs RTOS ได้
- ออกแบบ AI/Embedded pipeline แบบ end-to-end ได้
- วิเคราะห์ latency, power, performance และ deployment ได้
- เข้าใจ App Lab, Bricks, AI Models และวิธีคิดแบบ modular development

Suggested Next Topics

- Linux command line สำหรับ embedded developer
- Zephyr RTOS และ real-time scheduling
- OpenCV + Edge AI inference
- Docker Compose สำหรับ edge device
- ROS 2 สำหรับ robotics
- I2C/SPI/UART/CAN communication
- Model optimization เช่น quantization และ pruning



สไลด์หน้า 28: หน่วยงานและผู้ร่วมสนับสนุนจากสไลด์ต้นฉบับ

ข้อสรุปสุดท้าย

Arduino UNO Q คือสะพานระหว่างโลก Arduino แบบเดิมกับโลก Edge AI สมัยใหม่ คนที่เข้าใจบอร์ดนี้จะไม่ได้อัปเดตแค่เขียนโปรแกรมได้ แต่จะสามารถออกแบบระบบที่รับรู้โลกจริง ตัดสินใจ และควบคุมอุปกรณ์ได้อย่างมีประสิทธิภาพ